

1 Basic Definitions

Learning: Find a model f that maps from inputs to outputs given a labeled training dataset.

Inference: Use a model f to predict output y given new example $\tilde{x}$

Model: Function f mapping input $\tilde{x}$ to output prediction y .

Model family: Set of models that share the same structure but differ in parameters (e.g. layers in a NN)

Accuracy: $\frac{\sum_{i=1}^N \mathbb{I}[t^{(i)}=y^{(i)}]}{N}$

Parameter: Internal variable of model learned from data whilst training

Hyperparameter: Settings chosen before training that controls how the learning process works.

Parametric model: Any model with the parameter counts NEVER being $\Omega(N)$ (invariant to the data count)

1.1 KNN

Find $t^{(c)}$ corresponding to

$$\tilde{x}^{(c)} = \arg \min_{\tilde{x}^{(i)} \in \text{train set}} \text{distance}(\tilde{x}^{(i)}, \tilde{x})$$

Adv: Simple, no training, works for reg. and class, adapts easily to new data.

Disadvantage: Bad w/ high dims, sensitive to scaling, slow for large datasets, must store logs proportional to data. Make k odd or chose a tiebreaker.

1.2 Decision Trees

Large trees overfit, small trees underfit, trace the tree on inference.

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x),$$

$$H(X|Y=y) = -\sum_{x \in \mathcal{X}} p(x|Y=y) \log_2 p(x|Y=y).$$

Expected conditional entropy:

$$H(X|Y) = -\sum_{y \in \mathcal{Y}} p(y) \sum_{x \in \mathcal{X}} p(x|y) \log_2 p(x|y),$$

$$IG(X|Y) = H(X) - H(X|Y)$$

Properties: Entropy nonnegative, $H(Y|X) \leq H(Y)$, Y, X independent $\Rightarrow IG(Y|X) = 0$, fully determinant rids entropy.

Splitting: Keep splitting until all examples have same label, when no features left, when depth limit is reached (hyperparam), when loss stops decreasing enough (hyperparam)

1.3 Linear Regression

Loss measures how off **one feature** is, cost

measures **all**. $\mathcal{L}(y, t) = \frac{1}{2}(y - t)^2$,

$\mathcal{E}(\tilde{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y^{(i)}, t^{(i)})$. Hence, for linear model $\tilde{y} = W\tilde{x}$.

$$\mathcal{E}(\tilde{w}) = \frac{1}{2N} \sum_{i=1}^N (\tilde{w}^T \tilde{x}^{(i)} - t^{(i)})^2$$

$$\mathcal{E}(\tilde{w}) = \frac{1}{2N} (X\tilde{w} - \tilde{t})^T (X\tilde{w} - \tilde{t})$$

Gradients:

$$\frac{\partial \mathcal{E}}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (\tilde{w}^T \tilde{x}^{(i)} - t^{(i)}) x_j^{(i)}$$

$$\nabla_{\tilde{w}} \mathcal{E}(\tilde{w}) = \frac{1}{N} \sum_{i=1}^N (\tilde{w}^T \tilde{x}^{(i)} - t^{(i)}) \tilde{x}^{(i)}$$

$$\nabla_{\tilde{w}} \mathcal{E}(\tilde{w}) = \frac{1}{N} X^T (X\tilde{w} - \tilde{t})$$

$$\text{W/ bias: } \frac{\partial \mathcal{E}(\tilde{w})}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (\tilde{w}^T \tilde{x}^{(i)} + b - t^{(i)}) x_j^{(i)},$$

$$\frac{\partial \mathcal{E}(\tilde{w})}{\partial b} = \frac{1}{N} (X\tilde{w} + b - \tilde{t})$$

Regularizer: $\frac{\lambda}{2} \sum_{j=0}^D w_j^2, \nabla_{\tilde{w}} \left(\frac{\lambda}{2} \tilde{w}^T \tilde{w} \right) = \lambda \tilde{w}$

Stochastic GD: Randomly pick a training example, very noisy, $\tilde{w} \leftarrow \tilde{w} - \alpha \frac{\partial \mathcal{L}}{\partial \tilde{w}}$. Mini-batch GD falls in-between. Batch size is a hyperparameter

1.4 Logistic Regression

$$z = \tilde{w}^T \tilde{x}, y = \mathbb{I}[z \geq 0]$$

Loss functions:

- 0-1 $\mathcal{L}_{0-1}(y, t) = \mathbb{I}[y \neq t]$. Hard to optimize
- $\mathcal{L}_{SE}(z, t) = \frac{1}{2}(z - t)^2$. Large loss for v. correct
- $y = \sigma(z) = \frac{1}{1+e^{-z}}, \mathcal{L}_{SE}(y, t) = \frac{1}{2}(y - t)^2$, prediction always $[0, 1]$, ∇ small at v. large magnitudes
- Cross entropy:

$$\mathcal{L}_{CE}(y, t) = \begin{cases} -t \log(y) & t = 1 \\ -(1-t) \log(1-y) & t = 0 \end{cases}$$

CE loss derivation, $z = \tilde{w}^T \tilde{x}, y = \sigma(z) = \frac{1}{1+e^{-z}}$.

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)). \frac{\partial y}{\partial z} = y(1 - y).$$

$$\mathcal{L}_{CE}(y, t) = -t \log(y) - (1-t) \log(1-y).$$

GD update: $\mathcal{E}(\tilde{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{CE}(y^{(i)}, t^{(i)})$.

$$\begin{aligned} \frac{\partial \mathcal{L}_{CE}}{\partial w_j} &= \frac{\partial \mathcal{L}_{CE}}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_j} \\ &= \left(-\frac{t}{y} + \frac{1-t}{1-y} \right) y(1-y) x_j = (y-t)x_j \end{aligned}$$

Vectorized: $\tilde{w} \leftarrow \tilde{w} - \frac{\alpha}{N} X^T (X\tilde{w} - \tilde{t})$

1.5 Softmax Gradient

The softmax model is

$$\mathbf{z} = W\mathbf{x} \quad \mathbf{y} = \text{softmax}(\mathbf{z})$$

$$y_k = \frac{\exp(z_k)}{\sum_{m=1}^K \exp(z_m)}$$

$$\mathcal{L}(\mathbf{y}, \mathbf{t}) = -\mathbf{t}^T \log(\mathbf{y})$$

Derive: $\frac{\partial \mathcal{L}}{\partial \mathbf{w}_m}$. Derivation:

$$L = -\sum_{k=1}^K t_k \log(y_k)$$

$$\frac{\partial L}{\partial y_k} = -\frac{t_k}{y_k}$$

$$k \neq m \Rightarrow \frac{\partial y_k}{\partial z_m} = \frac{\partial}{\partial z_m} \left(\frac{\exp(z_k)}{\sum_{n=1}^K \exp(z_n)} \right)$$

$$= \frac{-\exp(z_k) \exp(z_m)}{\left(\sum_{n=1}^K \exp(z_n) \right)^2} = -y_k y_m$$

$$k = m \Rightarrow \frac{\partial y_k}{\partial z_k} = \frac{\partial}{\partial z_k} \left(\frac{\exp(z_k)}{\sum_{n=1}^K \exp(z_n)} \right)$$

$$= \frac{\exp(z_k) \left(\left(\sum_{n=1}^K \exp(z_n) \right) - \exp(z_k) \right)}{\left(\sum_{n=1}^K \exp(z_n) \right)^2}$$

$$= y_k (1 - y_k)$$

$$\frac{\partial y_k}{\partial z_m} = \begin{cases} -y_k y_m & \text{if } k \neq m \\ y_k (1 - y_k) & \text{if } k = m \end{cases}$$

$$\frac{\partial L}{\partial z_m} = \sum_{k=1}^K \frac{\partial L}{\partial y_k} \frac{\partial y_k}{\partial z_m}$$

$$= \sum_{k=1}^K \begin{cases} t_k y_m & \text{if } k \neq m \\ t_k (y_k - 1) & \text{if } k = m \end{cases}$$

$$= t_1 y_m + \dots + t_m (y_m - 1) + \dots + t_K y_m$$

Since $\mathbf{t}$ is one-hot: if $t_m = 1$, term left in sum is $t_m (y_m - 1) = y_m - 1 = y_m - t_m$. If $t_k = 1$ where $k \neq m$, then $t_m = 0 \Rightarrow t_k y_m = y_m = y_m - t_m$. Either way, we have that $\frac{\partial L}{\partial z_m} = y_m - t_m$. Hence:

$$\frac{\partial L}{\partial \mathbf{w}_m} = \frac{\partial L}{\partial z_m} \frac{\partial z_m}{\partial \mathbf{w}_m} = (y_m - t_m) \mathbf{x}$$

1.6 Softmax K=2 Proof

$$K = 2, y_k = \frac{e^{z_k}}{\sum_{m=1}^K e^{z_m}} \text{ correspond to } y = \frac{1}{1+e^{-z}}.$$

$$y_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{e^{z_1}}{e^{z_1} (1 + e^{z_2 - z_1})} = \frac{1}{1 + e^{-(z_1 - z_2)}}$$

1.7 NN Brute Forcing

$$[-N \mathbf{1}_{\text{TRUE}} + 0.5, -999, -999, 1, 1, -999]$$

1.8 The Vectorizer

Write the 1 vector in blackboard bold!

- $z_j^{(1)} = \sum_{i=1}^{D^{(0)}} W_{ji}^{(1)} a_i^{(0)} + b_j^{(1)}$
- $\mathbf{z}^{(1)} = W^{(1)} \mathbf{a}^{(0)} + \mathbf{b}^{(1)}$
- $\mathbf{Z}^{(1)} = A^{(0)} (W^{(1)})^T + \mathbf{1}_N (\mathbf{b}^{(1)})^T$
- $a_j^{(1)} = \text{ReLU}(z_j^{(1)})$
- $\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)}), A^{(1)} = \text{ReLU}(\mathbf{Z}^{(1)})$
- $a_k^{(2)} = \frac{e^{z_k^{(2)}}}{\sum_{k'=1}^{D^{(2)}} e^{z_{k'}^{(2)}}}$
- $\mathbf{a}^{(2)} = \frac{\exp(\mathbf{z}^{(2)})}{\mathbf{1}_{D^{(2)}}^T \exp(\mathbf{z}^{(2)})}$
- $A^{(2)} = \frac{\exp(\mathbf{Z}^{(2)})}{\exp(\mathbf{Z}^{(2)}) \mathbf{1}_{D^{(2)}}}$
- $\mathcal{L}(\mathbf{a}^{(2)}, \mathbf{t}) = -\sum_{k=1}^{D^{(2)}} t_k \log(a_k^{(2)})$
- $\mathcal{L}(\mathbf{a}^{(2)}, \mathbf{t}) = -\mathbf{t}^T \log(\mathbf{a}^{(2)})$
- $\mathcal{L} = -\frac{1}{N} \mathbf{1}_N^T (T \odot \log A^{(2)}) \mathbf{1}_{D^{(2)}}$
- $\frac{\partial \mathcal{L}}{\partial z_k^{(2)}} = a_k^{(2)} - t_k$
- $\nabla_{\mathbf{z}^{(2)}} \mathcal{L} = \mathbf{a}^{(2)} - \mathbf{t}$
- $\nabla_{Z^{(2)}} \mathcal{L} = \frac{1}{N} (A^{(2)} - T)$
- $\frac{\partial \mathcal{L}}{\partial w_{kj}^{(2)}} = \frac{\partial \mathcal{L}}{\partial z_k^{(2)}} a_j^{(1)}$
- $\nabla_{W^{(2)}} \mathcal{L} = (\nabla_{\mathbf{z}^{(2)}} \mathcal{L}) (\mathbf{a}^{(1)})^T$
- $\nabla_{W^{(2)}} \mathcal{L} = (\nabla_{Z^{(2)}} \mathcal{L})^T A^{(1)}$
- $\frac{\partial \mathcal{L}}{\partial a_j^{(1)}} = \sum_{k=1}^{D^{(2)}} \frac{\partial \mathcal{L}}{\partial z_k^{(2)}} W_{jk}^{(2)}$
- $\nabla_{\mathbf{a}^{(1)}} \mathcal{L} = (W^{(2)})^T (\nabla_{\mathbf{z}^{(2)}} \mathcal{L})$
- $\nabla_{A^{(1)}} \mathcal{L} = (\nabla_{Z^{(2)}} \mathcal{L})^T W^{(2)}$
- $\frac{\partial \mathcal{L}}{\partial z_j^{(1)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(1)}} \text{ReLU}'(z_j^{(1)})$
- $\nabla_{\mathbf{z}^{(1)}} \mathcal{L} = (\nabla_{\mathbf{a}^{(1)}} \mathcal{L}) \odot \text{ReLU}'(\mathbf{z}^{(1)})$
- $\nabla_{Z^{(1)}} \mathcal{L} = (\nabla_{A^{(1)}} \mathcal{L}) \odot \text{ReLU}'(\mathbf{Z}^{(1)})$

1.9 Vectorization Utilities

Sum $\rightarrow \leftarrow$ is $X \mathbf{1}_D$. Sum $\downarrow$ is $\mathbf{1}_N^T X$. $\mathbf{u} \odot \mathbf{v} = \text{diag}(\mathbf{u}) \mathbf{v}$.

Row-wise mult: $\text{diag}(\mathbf{u}) X$, col: $X \cdot \text{diag}(\mathbf{v})$;

$$\begin{bmatrix} -\mathbf{z}^T W - \\ - \\ - \end{bmatrix} = ZW$$

1.10 BV Decomposition

- Bias: model's inability to capture true relationship.
 - Variance: Model's sensitivity to random fluctuations. Resample from training set and model changes by a lot
 - Bayes error: Inherent noise in the problem
- First: $\mathbb{E}_{\mathbf{x}, t, \mathcal{D}} [(f(\mathbf{x}) - t)^2]$

$$\begin{aligned} &= \mathbb{E} \left[\left(f(\mathbf{x}) - \bar{f}(\mathbf{x}) + \bar{f}(\mathbf{x}) - t \right)^2 \right] \\ &= \mathbb{E} \left[\left(f(\mathbf{x}) - \bar{f}(\mathbf{x}) \right)^2 \right] + 0 + \\ &\quad \mathbb{E} \left[\left(\bar{f}(\mathbf{x}) - t \right)^2 \right] \end{aligned}$$

$$\begin{aligned} \text{Then: } \mathbb{E}_{\mathbf{x}, t, \mathcal{D}} \left[\left(\bar{f}(\mathbf{x}) - t \right)^2 \right] \\ &= \mathbb{E} \left[\left(\bar{f}(\mathbf{x}) - \bar{y}(\mathbf{x}) + \bar{y}(\mathbf{x}) - t \right)^2 \right] \\ &= \mathbb{E} \left[\left(\bar{f}(\mathbf{x}) - \bar{y}(\mathbf{x}) \right)^2 \right] + \\ &\quad \mathbb{E} \left[\left(\bar{y}(\mathbf{x}) - t \right)^2 \right] \end{aligned}$$

1.11 Bagging

Better if each random forest is uncorrelated.

$$\begin{aligned} V[y] &= V \left[\frac{1}{m} \sum_{i=1}^m y_i \right] \\ &= \frac{1}{m^2} V \left[\sum_{i=1}^m y_i \right] \\ &= \frac{1}{m^2} \sum_{i=1}^m V[y_i] = \frac{1}{m} V[y_i] \end{aligned}$$

1.12 MLE

$$\begin{aligned} p(x_j^{(i)} | c^{(i)}) &= \left(p(x_j^{(i)} | c^{(i)} = 1) \right)^{c^{(i)}} \times \\ &\quad \left(p(x_j^{(i)} | c^{(i)} = 0) \right)^{1-c^{(i)}} \\ \theta_{j1} &= \frac{\sum_{i=1}^N c^{(i)} x_j^{(i)}}{\sum_{i=1}^N c^{(i)}} \\ \theta_{j0} &= \frac{\sum_{i=1}^N (1 - c^{(i)}) x_j^{(i)}}{\sum_{i=1}^N (1 - c^{(i)})} \end{aligned}$$

MAP: Multiply by **every** possible parameter.

1.13 GDA

$$\mathcal{N}(\mathbf{x}; \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{D}{2}} |\Sigma|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} \tilde{\mathbf{x}}^T \Sigma^{-1} \tilde{\mathbf{x}} \right)$$

2 K-Means

Update assignments, update centers. D features, N data points, K clusters, $\tilde{\mathbf{m}}_k \in \mathbb{R}^D$ denotes the center of cluster k , $r_k^{(i)} \in \{0, 1\}$ denotes i is assigned to cluster k w/ center $\tilde{\mathbf{m}}_k$. Hence, $\tilde{\mathbf{r}}^{(i)}$ is one-hot. Global objective:

$$\mathcal{J}(\{\tilde{\mathbf{m}}_k\}, \{\tilde{\mathbf{r}}^{(i)}\}) = \min_{\{\tilde{\mathbf{m}}_k\}, \{\tilde{\mathbf{r}}^{(i)}\}} \sum_{i=1}^N \sum_{k=1}^K r_k^{(i)} \|\tilde{\mathbf{m}}_k - \tilde{\mathbf{x}}^{(i)}\|^2$$

Assignment step: for each point $\tilde{\mathbf{x}}^{(i)}$, find $\tilde{\mathbf{r}}^{(i)}$ (just brute-force):

$$\min_{\tilde{\mathbf{r}}^{(i)}} \sum_{k=1}^K r_k^{(i)} \|\tilde{\mathbf{m}}_k - \tilde{\mathbf{x}}^{(i)}\|^2$$

Center step (change centers): for center $\tilde{\mathbf{m}}_k$:

$$\arg \min_{\tilde{\mathbf{m}}_k} \sum_{i=1}^N r_k^{(i)} \|\tilde{\mathbf{m}}_k - \tilde{\mathbf{x}}^{(i)}\|^2$$

$$\tilde{\mathbf{m}}_k = \frac{\sum_{i=1}^N r_k^{(i)} \tilde{\mathbf{x}}^{(i)}}{\sum_{i=1}^N r_k^{(i)}}$$

We stop K-means when the assignments no longer change $\rightarrow$ we have converged to a local minimum. **Local minimum convergence is guaranteed:** each step never increases the objective, there's a finite no. of cluster assignments, algorithm must stop after finite no. of iterations. **If local minima is bad, restart with random cluster centers.**

3 EM

Soft K-means, clusters don't need to be a circle (allows elliptical clusters, and clusters are of a flexible size). A GMM represents $p(\tilde{\mathbf{x}})$ a mixture (weighted combination of multiple gaussian distributions) of K gaussian components:

$$p(\tilde{\mathbf{x}}) = \sum_{k=1}^K \pi_k \mathcal{N}(\tilde{\mathbf{x}} | \mu_k, \Sigma_k)$$

Where $\sum_{k=1}^K \pi_k = 1$, $\pi_k \geq 0$, $\forall k$

Generating data: pick $z^{(i)} \sim \text{Categorical}(\tilde{\pi})$, determine features $\tilde{\mathbf{x}}^{(i)}: \tilde{\mathbf{x}}^{(i)} | z^{(i)} \sim \mathcal{N}(\tilde{\mathbf{x}} | \tilde{\mu}_{z^{(i)}}, \tilde{\Sigma}_{z^{(i)}})$. Hence, generative model is $p(\tilde{\mathbf{x}}) = \sum_{k=1}^K \pi_k \mathcal{N}(\tilde{\mathbf{x}} | \mu_k, \Sigma_k)$. $z^{(i)}$ is a latent variable and indicates which Gaussian component generates $\tilde{\mathbf{x}}^{(i)}$.

EM Algorithm, D features, N data points, K clusters, params: $\tilde{\theta} = \{\pi_k, \tilde{\mu}_k, \tilde{\Sigma}_k, k = 1 \dots K\}$. Each cluster k

Has a mixing weight, mean, and covariance.

$r_k^{(i)} \in [0, 1]$ is the responsibility that components k takes for explaining point i (latent variable $z^{(i)}$ indicates that generated point i)

Max this likelihood:

$$\log p(\mathcal{D} | \tilde{\theta}) = \sum_{i=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\tilde{\mathbf{x}}^{(i)} | \tilde{\mu}_k, \tilde{\Sigma}_k) \right)$$

GMMs are universal density approximators. With MLE, training likelihood can grow unbounded, hence **there could be a tiny-variance gaussian on one training example**. This is overfitting. Hence, this is a two-step descent

Expectation step: Compute the responsibility each cluster k takes for explaining point i

$$r_k^{(i)} = p(z^{(i)} = k | \tilde{\mathbf{x}}^{(i)}, \tilde{\theta}) = \frac{p(z^{(i)} = k) p(\tilde{\mathbf{x}}^{(i)} | z^{(i)} = k)}{p(\tilde{\mathbf{x}}^{(i)})}$$

$$= \frac{\pi_k \mathcal{N}(\tilde{\mathbf{x}} | \tilde{\mu}_k, \tilde{\Sigma}_k)}{\sum_{k=1}^K \pi_k \mathcal{N}(\tilde{\mathbf{x}} | \tilde{\mu}_k, \tilde{\Sigma}_k)}$$

Maximization step: Maximizes the probability that each cluster generates the data points assigned to it

$$\begin{aligned} \pi_k &= \frac{1}{N} \sum_{i=1}^N r_k^{(i)}, \tilde{\mu}_k = \frac{\sum_{i=1}^N r_k^{(i)} \tilde{\mathbf{x}}^{(i)}}{\sum_{i=1}^N r_k^{(i)}} \\ \tilde{\Sigma}_k &= \frac{\sum_{i=1}^N r_k^{(i)} (\tilde{\mathbf{x}}^{(i)} - \tilde{\mu}_k) (\tilde{\mathbf{x}}^{(i)} - \tilde{\mu}_k)^T}{\sum_{i=1}^N r_k^{(i)}} \end{aligned}$$

4 PCA

- Projecting a 2D vector $\mathbf{x} \in \mathbb{R}^2$ onto $\mathbf{u} \in \mathbb{R}^2$:

$$\tilde{\mathbf{x}} = \underbrace{(\mathbf{x}^T \mathbf{u})}_{\text{mag}} \cdot \underbrace{\mathbf{u}}_{\text{dir}}$$

- The **representation** of $\mathbf{x}$ in basis U is $\mathbf{z} = U^T \mathbf{x}$
- Reconstruction error $\|\mathbf{x} - \tilde{\mathbf{x}}\|^2$

- Before PCA, data is best centered since variance is about how data varies around mean, not origin. $\tilde{\mathbf{x}}^{(i)} = \mathbf{x}^{(i)} - \mu$
 - Optimal basis (both are equivalent):
 - Max recons. Var.: $\max \frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}}^{(i)}\|^2$
 - Min recons. Err.: $\min \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2$
- Why are the two conditions above equivalent? First, $\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}$ and $\tilde{\mathbf{x}}^{(i)}$ are orthogonal. Then: $\|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2 + \|\tilde{\mathbf{x}}^{(i)}\|^2 = \|\mathbf{x}^{(i)}\|^2$. Averaging over data points, we obtain:

$$\frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2 + \frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}}^{(i)}\|^2 = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)}\|^2$$

4.1 Computing PCA Subspace

Given $\mathbf{x}^{(i)} \in \mathbb{R}^D$, compute $\mathbf{z}^{(i)} \in \mathbb{R}^K$, $K < D$ that preserves as much variance as possible.

- Center data: $\tilde{\mathbf{x}}^{(i)} = \mathbf{x}^{(i)} - \mu$
- Compute cov: $\Sigma = \frac{1}{N} \sum_{i=1}^N (\tilde{\mathbf{x}}^{(i)}) (\tilde{\mathbf{x}}^{(i)})^T$
 - Cov mat. Captures how much data varies in each direction & how variations in diff. dims relate to each other
- Find principal components: decompose $\Sigma = Q \Lambda Q^T$, $Q = [\mathbf{u}_1, \mathbf{u}_2 \dots]$ (eigenvectors), $\Lambda = \text{diag}(\lambda_1, \dots)$ (eigenvalues). Σ is PSD $\Rightarrow$ eigenvals real, NN, $Q^{-1} = Q^T$. Then: sort eigenvalues in descending order, select top k corresponding eigenvectors to form $U_k = [\mathbf{u}_1 \dots \mathbf{u}_k]$. U_k span the subspace that maxes proj. var or min. recons err

4.2 Shall Data Be Normalized

(For variance only, always normalize the mean). DO when: Features have diff. units, features have vastly diff. scales, want equal influence from all features

DON'T when: all features same unit, variance differences are meaningful

5 Ethics

COMPAS is the recidivism prediction tool. Fairness:

- Individual fairness:** fair if it treats people who are **relevantly similar**, similarly
- Group fairness:** different demographic groups should receive similar overall outcomes
 - e.g. fair if men/women have similar TPR or FPR
- Relevant similar:** Probably intent, need, consent, risk. Unlikely: race, ethnicity, arbitrary traits.

Sycophancy and Manipulation

- Noggle:** Manipulative action is the intentional attempt to get someone's **beliefs, desires, or emotions** to violate **their** norms or ideals, from the perspective of the manipulator. (*Influence* aimed at making people act against what they themselves think is right)
- Sussur, Roessler, Nissenbaum (SNR):** Causing someone to make non-ideal decisions is sometimes manipulative, but it is not manipulative in every case. Manipulation is imposing a hidden or covert influence on another person's decision-making. *i.e.* influence $\neq$ manipulation unless emotional pressure, deceptive framing, or psych. Tactics are used
- SNR is about the mechanism (is it hidden, does it bypass awareness)? Noggle is about the goal: is the influencer trying to steer someone away from their own values?